Prog II:
Aula de abertura

Profª. Tainá Isabela

Apresentação

Acadêmica:

- Técnico em Informática - IFMT
- Bacharel em Ciências da Computação - UFMT
- Pós em BI, Big Data e Inteligência Artificial

Profissional:

- Desenvolvedora Back-end desde 2021;

Plano de Ensino

Ementa

- Estruturas fundamentais de programação em C++: Entrada/Saída; estruturas de controle, tipos de dados simples e estruturados, alocação dinâmica de memória e ponteiros inteligentes.
- Conceitos de orientação a objetos em C++: Classes/Objetos;
- Atributos e métodos;
- Encapsulamento;
- Construtores e destrutores;
- Polimorfismo e sobrecarga de operadores;
- Herança;
- Funções virtuais e amigas e Classes abstratas;
- Compilação e execução;
- Espaço de nomes;
- Streams;
- Tratamento de Exceções.

Avaliação

Questionários e Exercícios (Peso 3,0)

Atividades ao longo do curso para fixação do conteúdo.

Trabalho Prático (Peso 3,0)

Trabalhos (T1 e T2) envolvendo atividades aplicadas, estudos dirigidos, projetos ou produções correlatas aos conteúdos ministrados;

Prova (Peso 4,5)

Provas (P1 e P2) destinadas à verificação do domínio dos conteúdos teóricos e práticos abordados na disciplina;

Bibliografia

- Mendes,A; 2010; Introdução à Programação Orientada a Objetos com C++; Campus;

- Mizrahi,V.V.; 2005; Treinamento em Linguagem C++, modulo 1, 2a edição; Makron Books;

- Mizrahi,V.V.; 2005; Treinamento em Linguagem C++, modulo 2, 2a edição; Makron Books;

Prog II: Introdução à linguagem, estrutura e execução

Profª. Tainá Isabela

Histórico

O C++ surgiu como uma evolução da linguagem C.

- Criado por **Bjarne Stroustrup;**
- Desenvolvido na década de 1970–1980;
- Inicialmente chamado de “C com classes”;

Histórico

A Linguagem C foi inventada e foi implementada no início dos anos 70 por Dennis Ritchie em um DEC PDP-11, usando o Sistema Operacional UNIX.

A linguagem C se tornou uma das linguagens de programação mais utilizada, por ser flexível e ainda poderosa, sendo que ela é a responsável pela criação de alguns software famosos e a base de outros como jogos.

Porém a linguagem C encontra seus limites quando o tamanho de um projeto ultrapassa certo ponto de **25.000 a 100.00 linhas de código**.

Histórico

Como o Unix era escrito em C, deveria-se manter a compatibilidade, ainda que adicionando novos recursos.

Alguns dos desafios incluíam simular a infraestrutura da comunicação entre processos num sistema distribuído ou de memória compartilhada e escrever drivers para tal sistema.

Stroustrup percebeu que a linguagem **Simula 67** possuía características bastante úteis para o desenvolvimento de software, mas que era muito lenta para uso prático.

Por outro lado, a linguagem **BCPL** era rápida, mas era muito baixo nível, dificultando sua utilização no desenvolvimento de aplicações.

Histórico

A partir de sua experiência de doutorado, começou a acrescentar elementos do **Simula 67** no C, especialmente os recursos de criação e manipulação de objetos.

O C foi escolhido como base de desenvolvimento da nova linguagem pois possuía uma proposta de uso genérico, era rápido e também portável para diversas plataformas.

Algumas outras linguagens que também serviram de inspiração para o cientista da computação foram ALGOL 68, Ada, CLU e ML.

Histórico

Ainda em 1983 o nome da linguagem foi alterado de **C with Classes** para **C++**.

Antes implementada usando um pré-processador, a linguagem passou a exigir um **compilador próprio**, escrito pelo próprio Stroustrup.

Novas características foram adicionadas, como funções virtuais, **sobrecarga de operadores** e funções, **referências**, **constantes**, **gerenciamento manual de memória**, melhorias na verificação de tipo de dado e estilo de comentário de código de uma linha *(//)*.

Histórico

Em 1985 foi lançada a primeira edição do livro The C++ Programming Language, contendo referências para a utilização da linguagem, já que ainda não era uma norma oficial. A primeira versão comercial foi lançada em outubro do mesmo ano.

Em 1989 a segunda versão foi lançada, contendo novas características como **herança múltipla**, **classes abstratas**, **métodos estáticos**, **métodos constantes** e membros protegidos, incrementando o suporte a **orientação a objeto**.

Em 1990 foi lançado o livro The Annotated C++ Reference Manual, que tornou-se base para o futuro padrão. Outras adições na linguagem incluem gabaritos, **tratamento de exceções**, espaço de nomes, **conversão segura de tipo de dado** e o tipo booleano.

Histórico

Assim como a linguagem, sua biblioteca padrão também sofreu melhorias ao longo do tempo. Sua primeira adição foi a biblioteca de E/S, e posteriormente a Standard Template Library (STL);

Ambas tornaram-se algumas das principais funcionalidades que **distanciaram a linguagem em relação a C**.

Criada primordialmente na HP por Alexander Stepanov no início da década de 1990 para explorar os potenciais da programação genérica, a STL foi apresentada a um comitê unificado ANSI e ISO em 1993 à convite de Andrew Koenig. Após uma proposta formal na reunião do ano seguinte, a biblioteca recebe o aval do comitê.

Histórico

Depois de anos de trabalho, o mesmo comitê ANSI/ISO padronizou o C++ em 1998 (ISO/IEC 14882:1998). Após alguns anos foram reportados defeitos e imprecisões no documento, e uma correção foi lançada em 2003.

Por muito tempo, o **C++ foi encarado como um superconjunto do C**. Entretanto, em 1999 o novo padrão ISO para a linguagem C tornou as duas linguagens ainda mais **diferentes entre si**. Devido a essas incompatibilidades, muitas empresas que desenvolvem compiladores não oferecem suporte à versão mais recente da linguagem C.

Pode-se dizer que C++ foi a única linguagem entre tantas outras que obteve sucesso como uma sucessora à linguagem C, inclusive servindo de inspiração para outras linguagens como Java, a IDL de CORBA e C#.

Etimologia

Durante sua fase inicial de desenvolvimento, a linguagem era chamada "novo C", "C84" ou ainda "C com classes". O termo "C++" é creditado a Rick Mascitti, e foi utilizado pela primeira vez em dezembro de 1983. O termo é uma referência ao operador de incremento ++, significando um **acréscimo** (uma evolução) à linguagem C.

Características

No livro ***In The Design and Evolution of C++*** (1994), Bjarne Stroustrup descreve algumas regras que ele utiliza para desenvolver a linguagem, como exemplificado abaixo:

- C++ é desenvolvido para ser uma linguagem tipada estaticamente e de propósito geral que é tão eficiente e portátil quanto o C.
- C++ é desenvolvido para suportar múltiplos paradigmas.
- C++ é desenvolvido para fornecer ao programador escolhas, mesmo que seja possível ao programador escolher a opção errada.

Características

- C++ é desenvolvido para ser o mais compatível com C possível, fornecendo transições simples para código C.
- C++ evita fornecer facilidades que são específicas a certas plataformas ou a certos grupos de desenvolvedores.
- C++ não exige overhead para facilidades que não são utilizadas.
- C++ é desenvolvido para ser utilizado mesmo sem um ambiente de desenvolvimento sofisticado.

Características Principais

- Linguagem compilada;
- Tipagem estática;
- Alto desempenho;
- Muito usada em:
  - Jogos;
  - Sistemas operacionais;
  - Sistemas embarcados;

Estrutura de um Programa em C++

```
#include <iostream>

using namespace std;

int main() {
    cout << "Olá, mundo!" << endl;
    return 0;
}
```

#include <iostream>

- Biblioteca de entrada e saída;
- Permite usar cout;

Estrutura de um Programa em C++

```
#include <iostream>

using namespace std;

int main() {
    cout << "Olá, mundo!" << endl;
    return 0;
}
```

using namespace std;

- Evita escrever std::cout;
- Simplifica o código;

Estrutura de um Programa em C++

```
#include <iostream>

using namespace std;

int main() {
    cout << "Olá, mundo!" << endl;
    return 0;
}
```

int main()

- Função principal;
- Ponto de entrada do programa;

Estrutura de um Programa em C++

```
#include <iostream>

using namespace std;

int main() {
    cout << "Olá, mundo!" << endl;
    return 0;
}
```

{ }

- Delimitam o bloco de código;

Estrutura de um Programa em C++

```
#include <iostream>

using namespace std;

int main() {
    cout << "Olá, mundo!" << endl;
    return 0;
}
```

cout << "Olá, mundo!"

- Saída no console;

Estrutura de um Programa em C++

```
#include <iostream>

using namespace std;

int main() {
    cout << "Olá, mundo!" << endl;
    return 0;
}
```

<< endl;

- Faz o cursor ir para a próxima linha, igual ao \n.
- Força a saída a aparecer imediatamente (flush);

Estrutura de um Programa em C++

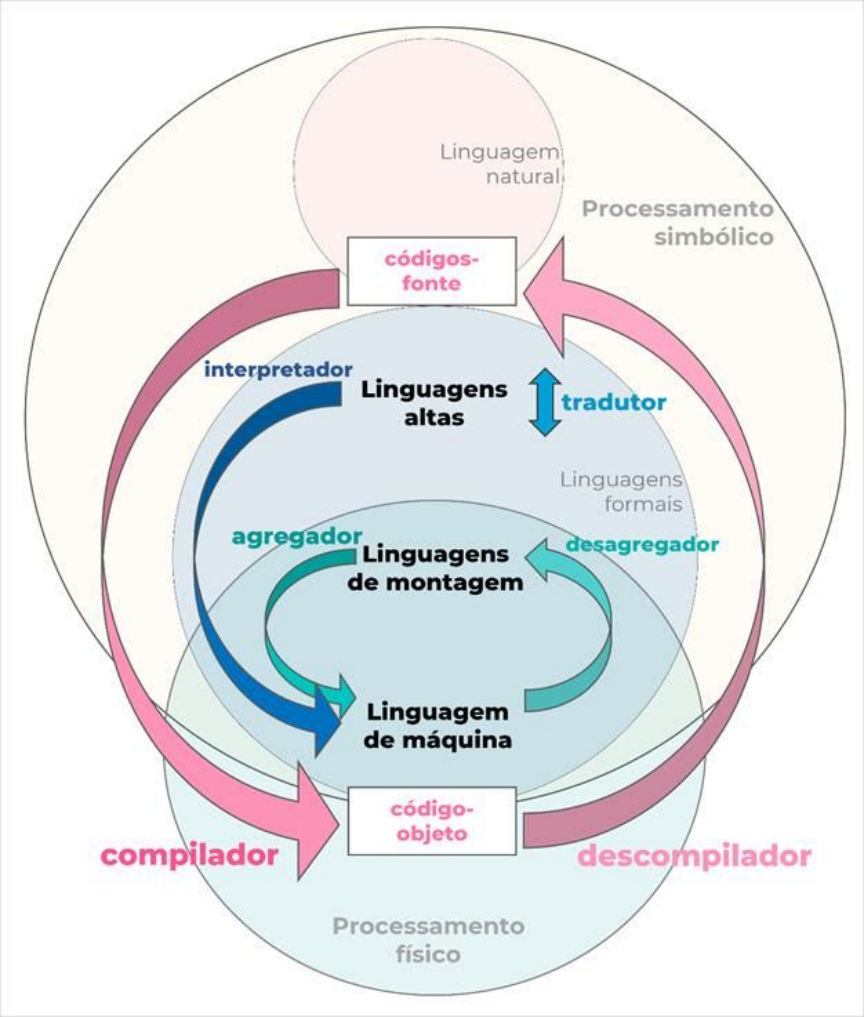
```
#include <iostream>

using namespace std;

int main() {
    cout << "Olá, mundo!" << endl;
    return 0;
}
```

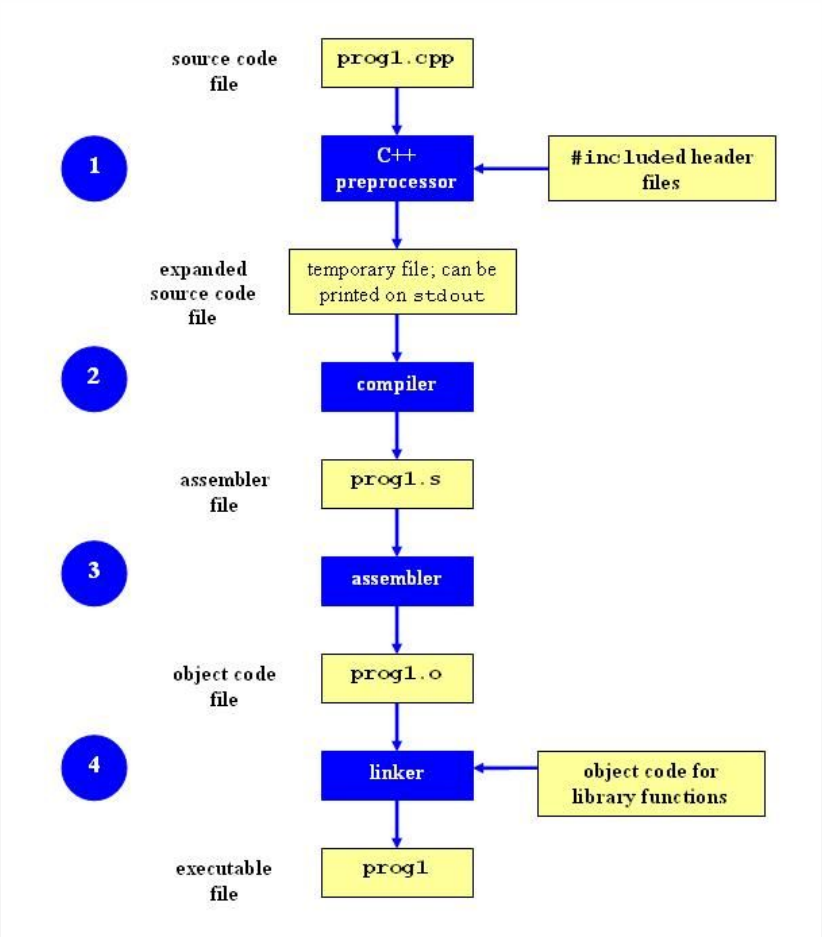
return 0;

- Indica execução bem-sucedida;

Observações

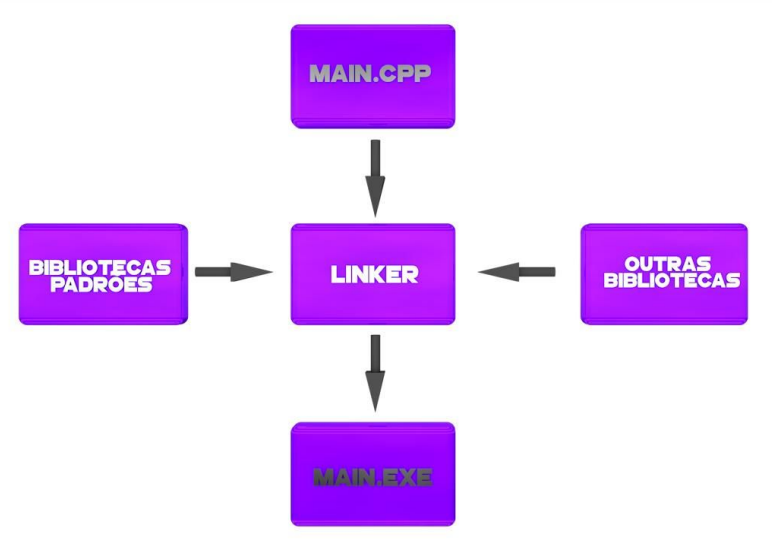
- Cada instrução termina com ;
- C++ diferencia maiúsculas de minúsculas (Case sensitive);
- Organização e indentação importam;


Linguagem natural
Processamento simbólico
códigos-fonte
interpretador
Linguagens altas
tradutor
Linguagens formais
agregador
Linguagens de montagem
desagregador
Linguagem de máquina
código-objeto
compilador
descompilador
Processamento físico


source code file
prog1.cpp
1
C++ preprocessor
#included header files
expanded source code file
temporary file; can be printed on stdout
2
compiler
assembler file
prog1.s
3
assembler
object code file
prog1.o
4
linker
object code for library functions
executable file
prog1

Compilação e Execução

C++ é uma **linguagem compilada**, diferente de Python.




```
thuustalu@freneskae:~/comprog/lec/cli$ g++ program.cpp -o program
thuustalu@freneskae:~/comprog/lec/cli$ ./program
1331 6
1337
thuustalu@freneskae:~/comprog/lec/cli$
```

Compilação e Execução

Etapas:

1. Código fonte (.cpp);
2. Compilação;
3. Geração de executável;
4. Execução;

Compilar:

```
g++ programa.cpp -o programa
```

Executar:

```
./programa
```

Erros comuns

- Erros de sintaxe;
- Arquivo não encontrado;
- Falta de compilador;

Qual a diferença entre erro de compilação e erro de execução?

Erro de Compilação vs Erro de Execução

1. Erro de Compilação

   - Acontece ANTES do programa rodar
   - São erros que impedem o código de ser transformado em executável pelo compilador (g++).

     Tipos comuns:

     - Falta de ;
     - Variável não declarada
     - Sintaxe incorreta
     - Biblioteca faltando

Problema na escrita do programa

Erro de Compilação vs Erro de Execução

2. Erro de Execução (Runtime)
   - O programa compila normalmente mas quebra durante a execução;

     Tipos comuns:

     - Divisão por zero;
     - Acesso inválido à memória (segmentation fault);
     - Entrada inválida do usuário;
     - Arquivo não encontrado;

     Problema na lógica do programa

Função main

```
int main() {
        return 0;
}

int main(int argc, char* argv[]) {
   return 0;
}
```

Podemos ter mais de uma função main?

- argc → quantidade de argumentos;
- argv → vetor de argumentos;

Questionário

Exercícios

- ★ Exercício 1

  Escreva um programa que mostre:

  Bem-vindo ao C++!

- ★ Exercício 2

  Modifique o programa para exibir seu nome.

Instalar gcc (linux): **sudo apt install build-essential**